

1.1 LoRa

LoRa, which stand for Long Range, is a wireless modulation technique based on the Chirp Spread Spectrum(CSS) technology. It works by sweeping the signal between a lower and an upper frequency over a certain time period, similar to how dolphins and bats communicate. These sweeps are called chirp pulses.

In order to setup the LoRa modulation you should specify the spreading factor(SF) and the bandwidth. A larger bandwidth will allow for more data to be send but is limited by law. For the sub-gigahertz bands, 915 MHz, 868 MHz and 433 MHz, in europe the bandwidth can be 125 kHz or 250 kHz. In the rest of the world 500 kHz can also be used. LoRa can also be used in the 2.4 GHz band in which case there is an additional option of 1625 kHz. This 2.4 GHz band will allow for higher data rates but will also have a shorter range.

The spreading factor can be between 7 and 12. A higher value will increase resistance to interference and increase the distance over which data can be send but will decrease the data rate.

One important fact about LoRa is that the on air time is limited by law to 1%. This means you cannot send for more then 1% of the time or 36 seconds per hour. This is to prevent the LoRa network from being flooded by a single device. This also means we cannot send large amounts of data over LoRa but rather use it to send small periodic messages. This restriction does not apply to the 2.4 GHz band.

1.1.1 WHY LoRa

you might wonder why we use LoRa if it has all these limitations. To understand this better we take a look at fig. 1.1 which shows LoRa's use case in a graph.

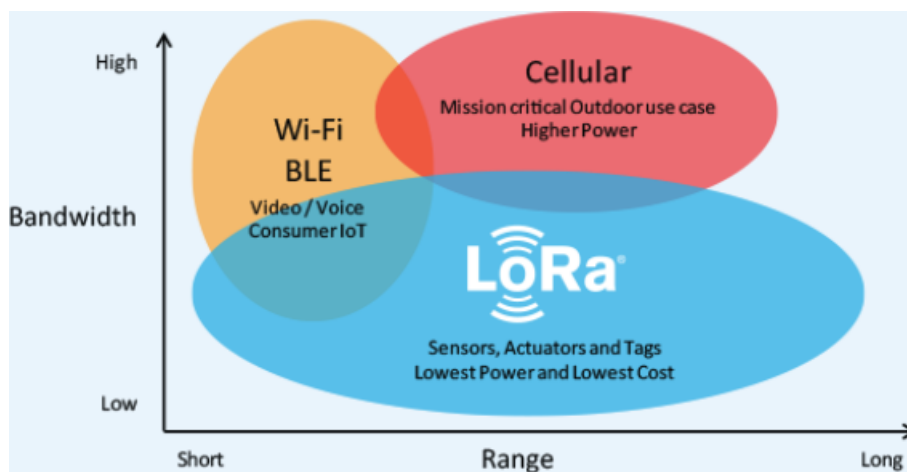


Figure 1.1: LoRa use case, taken from *Benefits of LoRa* / Semtech 2023

As we can see all known wireless technologies focus on high data rate while there is a large space for low data rate, long range communication. These technologies also often require a lot of power which is not desirable on battery operated nodes. This is where LoRa comes in as a low data rate, low power, long range communication technology.

1.1.2 LoRaWAN

LoRaWAN is a Low Power Wide Area Network(LPWAN) protocol which defines the communication protocol and the system architecture. The protocol and network architecture are responsible for things like keeping the energy usage low in order to preserve battery life, network capacity, quality of service and security.

1.1.3 LoRaWAN NETWORK ARCHITECTURE

Usually when building a network we build a mesh network where end-nodes are also responsible for forwarding packets from other nodes. In case of LoRaWAN this is not the case. Due to the long range of LoRa and the need to keep the energy usage low we build a long range star architecture. An example of a LoRaWAN network can be seen in fig. 1.2.

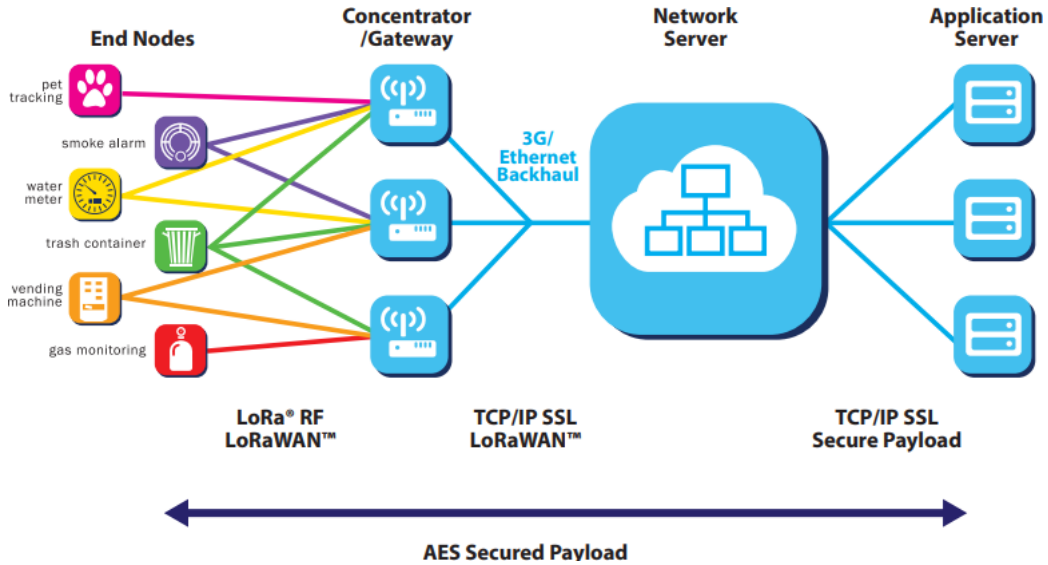


Figure 1.2: LoRaWAN example network, taken from Alliance, 2015

Nodes part of a LoRaWAN network don't unicast their packet to a specific gateway but instead broadcast them so multiple gateways receive the packet. The gateways will then each forward the packet to the network server where more complex logic is applied. This way the complexity of the network itself stays limited and it increases the chance of a successful transmission. It also prevents the need of a handover protocol for mobile nodes which is yet another way LoRaWAN keeps the network simple and energy efficient.

LoRaWAN uses an ALOHA like MAC protocol which means nodes can send whenever they want to. This prevents the need for nodes to wake up periodically to synchronize.

1.1.4 LoRaWAN CAPACITY

The gateways in a LoRaWAN network need to be very powerful in order to be able to process all the messages at once. 1 of the ways this is achieved is by using the fact that LoRa is a spread spectrum modulation technique. This means different spreading factors will be orthogonal to

each other. This allows a gateway to receive multiple packets on the same frequency at different data rates. Nodes also have adaptive data rate which allows nodes close to the gateways to send at a higher data rate for a shorter period and thus allowing the gateway to process more packets.

1.1.5 LoRaWAN DEVICE CLASSES

Since not all devices have the same requirements LoRaWAN has divided them into 3 classes:

- Class A or battery powered sensors: these devices are the most energy efficient, must be supported by all devices and only allow downlink after a sensor Tx. Each end-device's uplink transmission is followed by 2 short receive windows.
- Class B or battery powered actuators: these devices are less energy efficient but also experience lower latency. They support slotted communication synchronization with a beacon. Class B supports the same receive windows as class A but also has additional scheduled receive windows due to these synchronized beacons.
- Class C or main powered actuators: These devices do not run on battery so they can listen continuously. They also experience no latency for downlink messages.

1.1.6 LoRaWAN SECURITY

While security and IoT are a difficult combination due to the fact that IoT devices are often resource constrained, LoRaWAN has a 2 of security layers. 1 for the network and 1 for the application. The network layer is responsible for authenticating the end-node while the application layer is responsible for keeping the user's application data secure from the network operator. For this to work, LoRaWAN makes use of AES encryption where the key exchange happens with an IEEE EUI64 identifier.

1.1.7 LoRa 2.4 GHz

LoRa 2.4 GHz is a new version of LoRa which uses the 2.4 GHz band instead of the sub-gigahertz bands. This allows for higher data rates. While it does have shorter range, the range is still larger than other communication technologies. Another advantage is that there is no duty cycling restriction. This means we can send for more than 1% of the time. This allows us to send larger amounts of data.

The final advantage is that manufacturers can now make 1 chip that will work in all regions instead of the current situation where every region has different LoRa bands with each their own problems.

1.2 RDC PROTOCOLS

RDC or Radio Duty Cycling protocols are used to reduce the power consumption of a wireless device. They do this by turning off the radio when it is not needed. In theory a radio should only be on when sending a packet or when receiving one. The problem is that we do not know when a packet will be sent and thus need some kind of logic to allow communication.

There are 2 types of RDC protocols, low-power listening and low-power probing. In low-power listening the receiver periodically wakes up and listens to the medium. If the sender wants to send a packet the receiver will stay awake till the packet is received.

In low-power probing, the receiver will periodically wake up and send out a probe. If the sender wants to send a packet it will be awake and receive this probe. The sender will then send the packet.

We will now take a more in depth look at the 3 RDC protocols which are implemented in contiki-os.

1.2.1 LOW-POWER PROBING

The first protocol we will look at is Low-Power Probing or LPP. There are always 2 scenarios we need to look at, broadcast and unicast.

1.2.1.1 Unicast

In unicast LPP the receiver will wake up periodically and send out a probe. If the sender doesn't want to send a packet it will be asleep and not receive the probe. In this case the receiver will go back to sleep.

If the sender wants to send a packet it will be awake and receive the probe. It will then transmit the packet, wait for an ACK and go back to sleep. The receiver will also go back to sleep after sending the ACK.

It is important to note the sender will only be awake for 1 period if he wants to send a packet before going back to sleep. This is to prevent the sender staying awake forever when trying to reach a broken node.

The entire sequence can be visualized in fig. 1.3.

In order to correctly understand the image we need to know the following:

- the green box means Rx is on and Tx is off
- the red box means Rx and Tx are on

This will continue to be true for the rest of the images.

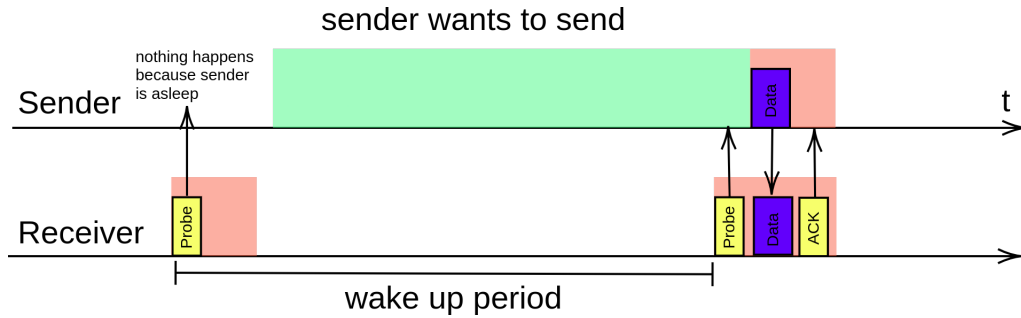


Figure 1.3: LPP unicast

1.2.1.2 Broadcast

In case of broadcast LPP there is only 1 difference: the sender does not go back to sleep after sending 1 packet but instead always stays awake for the entire period. This guarantees that every node has a chance to probe the sender in order to receive the packet. This can be visualized in fig. 1.4.

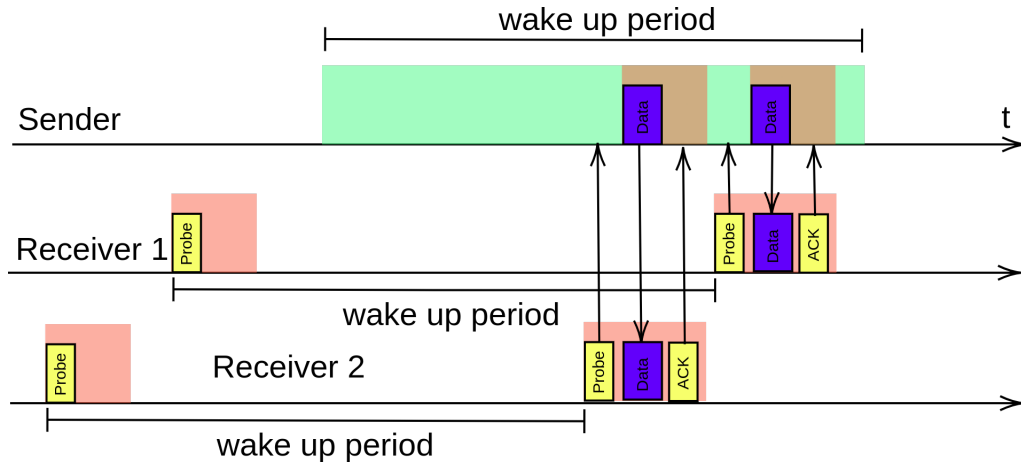


Figure 1.4: LPP broadcast

1.2.2 X-MAC

The next RDC protocol we will look at is X-MAC which is of the low-power listening type.

1.2.2.1 Unicast

This protocol is the opposite of LPP, instead of the receiver probing the sender, the sender will probe the receiver. The receiver will still be the one waking up once every period while the sender will continue to probe the entire period or until the packet is send. this can be seen in fig. 1.5.

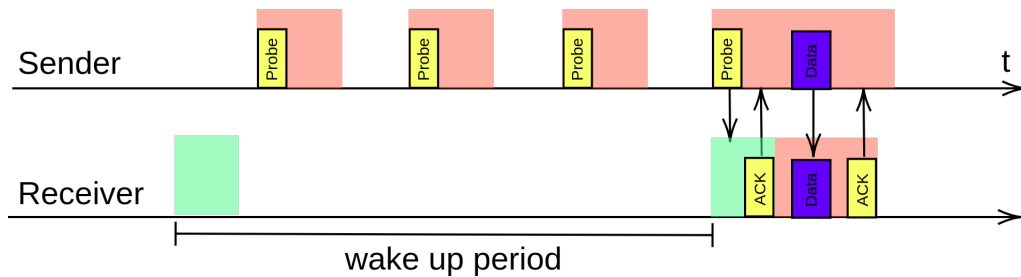


Figure 1.5: X-MAC unicast

1.2.2.2 Broadcast

The broadcasting version of X-MAC is a bit different. Instead of probing the receiver to see if its awake, the sender will broadcast the packet continuously for 1 whole period. When the receiver wakes up it will then detect radio activity and stay awake until it has received 1 whole packet. This can be seen in fig. 1.6.

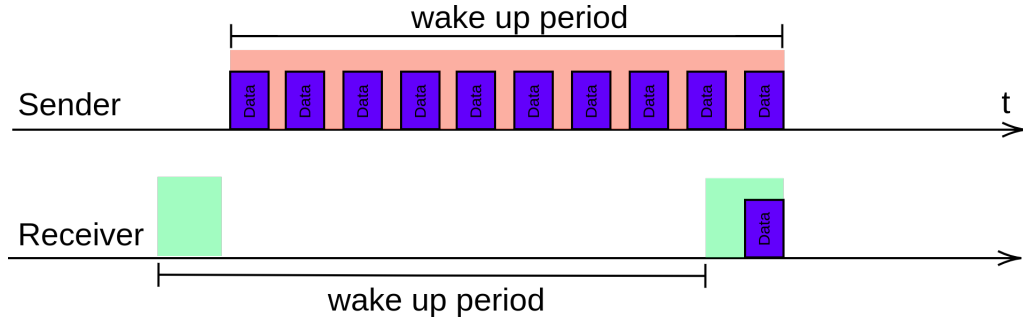


Figure 1.6: X-MAC broadcasting

1.2.3 CONTIKI-MAC

The last protocol we will look at is contiki-MAC which is also of the low-power listening type, like X-MAC, but is more complex in order to reduce the latency of the protocol. One of the biggest difference is that we make use of a double CCA (clear channel assessment) which is an energy efficient way of determining whether there is activity.

1.2.3.1 Unicast

Unicast contiki-MAC is similar to X-MAC with the difference that the sender won't send probes but will send the packet as a sort of probe. This way the receiver can use a double CCA to determine whether there is activity and instantly receive the next packet without the need to send an ACK. After receiving the packet the receiver will send an ACK to the sender and both will go back to sleep. The sender will transmit until either an ACK is received or 1 whole period has passed. This can be seen in fig. 1.7 where the purple lines represent a CCA.

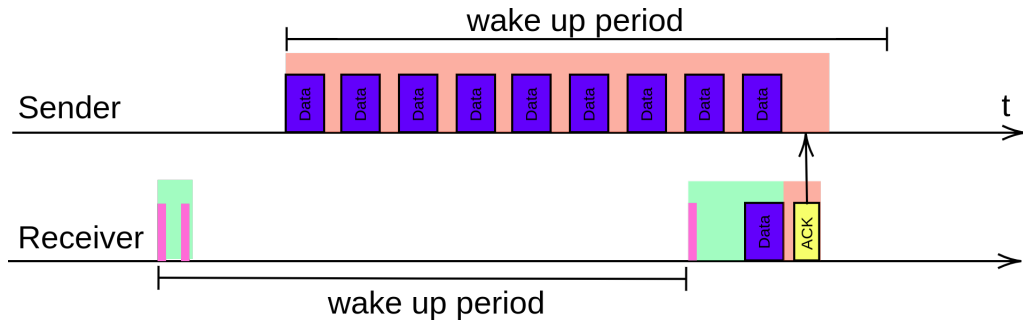


Figure 1.7: contiki-MAC unicast

In order for this sequence to work correctly, the double CCA needs to follow some timing restrictions which are discussed in 1.2.3.4.

1.2.3.2 Broadcast

Broadcast contiki-MAC follows the same logic as broadcast X-MAC with the only addition being the use of a double CCA by the receiver. This can be seen in fig. 1.8.

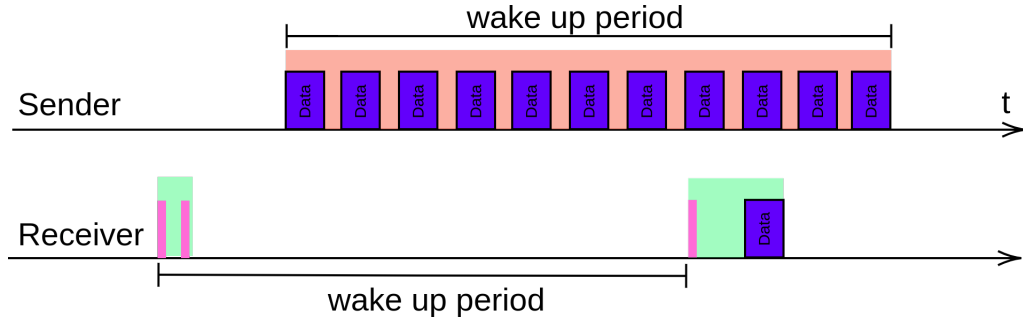


Figure 1.8: contiki-MAC broadcast

1.2.3.3 Clear Channel assesment

A CCA determines whether there is activity by measuring the Received Signal Strength Indicator (RSSI). If the RSSI is above a certain threshold the channel is considered busy and the receiver will wait for the next packet to be transmitted. In case the RSSI is below the threshold the channel is considered idle and the receiver will wait a very short time and try a second CCA. If the second CCA is also negative the receiver will go back to sleep. The time between 2 CCA's has to be larger than the time between 2 packets being sent. This is because each node has its own clock with its own clock drift. If the time between 2 CCA's is shorter than the time between 2 packets there is a possibility of a packet being transmitted but not being noticed by the receiver. This can be seen in 1.9.

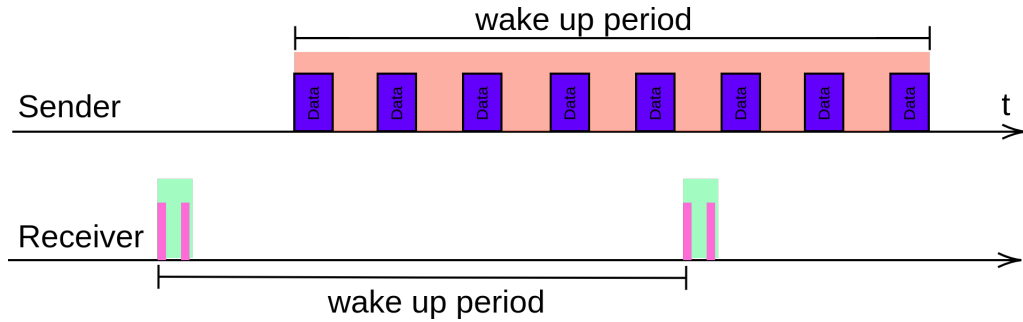


Figure 1.9: example of double CCA failing without the time restriction

1.2.3.4 Time restrictions

contiki-MAC has a couple more time restrictions which need to be followed. To note them we will use the following variables:

- t_i the interval between each packet transmission
- t_r the time required for a stable RSSI, needed for a stable CCA indication
- t_c the interval between each CCA
- t_a the time between receiving a packet and sending the ACK
- t_d the time required for successfully detecting an ACK from the receiver
- t_s the time required to send the shortest possible packet

the first restriction we already discussed in 1.2.3.3. It states that $t_c > t_i$.

The second restriction follows on the first one and states that there has to be a minimum packet size. this can be written as $t_s > t_r + t_c + t_r$.

After a CCA is successful the receiver stays on until a full packet is received. After this an ACK is sent. This has to be done before the sender transmits the next packet. This means $t_i > t_a + t_d$.

If we put all these restrictions together we get the following timing restriction equation:

$$t_s > t_c + 2t_r > t_c > t_i > t_a + t_d.$$

Of course if we want to implement this in code we need to know the values of these constants. To determine their value we can look at the IEEE 802.15.4 standard which has a couple of rules we need to follow.

The first rule we must follow is $t_a = 12$ symbols. in 802.15.4 1 symbol equals 4/250 ms which means $t_a = 48/250$ ms = 0.192 ms.

The next variable we can determine is t_d since we know the receiver can reliably detect the ACK after a 4 bytes preamble and a 1 byte start delimiter. This gives us $t_d = 40/250$ ms = 0.16 ms.

The following constant t_d is radio specific. In case of the CC2420 radio $t_d = 0.192$ ms.

If we fill the previous constants into the time restriction equation we get $t_s > t_c + 0.384$ ms $> t_c > t_i > 0.352$ ms. We can then calculate the minimum packet send time to be $t_s = 0.736$ ms.

If we choose a bitrate of 250 kb/s this means a packet must be 23 bytes including preamble, start of packet delimiter, length field and 16 bytes for data. In order to guarantee the packet size, packet padding can be added. In case we work with IPv6 the packet will always be larger than the minimum packet size and this restriction can be ignored. Although if 6LoWPAN is used there might be IPv6 header compression which does not guarantee the minimum packet size. In this case padding should be added.

In contiki-OS the remaining constants are set to $t_i = 0.4$ ms, $t_c = 0.5$ ms, $t_s = 0.884$ ms.

1.2.3.5 packet detection and fast sleep

One of the features making contiki-MAC superior to X-MAC is packet detection and fast sleep. After having a successful CCA the simple way to determine how long the receiver should stay awake is for $t_l + t_i + t_l$ which is the time to send the largest allowed packet + the time between 2 packets + again the time to send the largest allowed packet. This is a very conservative approach and will result in a lot of wasted energy. contiki-MAC fast sleep uses additional rules to determine how long the receiver should stay awake.

- if the CCA is successful but the signal is longer than t_l this means there is noise being detected instead of a packet. The receiver goes back to sleep.
- if the CCA is successful and the silence after the initial signal is larger than t_i we are again detecting noise instead of a packet. The receiver goes back to sleep.
- if the CCA is successful followed by an allowed silence period but the following signal does not qualify as a packet, we are receiving noise. The receiver goes back to sleep.

1.2.3.6 Transmission phase-lock

Once the sender has sent a successful packet to the receiver it can then use that information to optimize when to send. By receiving an ACK from the receiver the sender can estimate when the receiver will wake up again and instead of sending for an entire period, only start sending when it thinks the receiver is about to wake up.

1.3 CONTIKI-NG

contiki-ng is an open source, cross platform operating system for the Internet of Things. There are many implemented standard protocols such as IPv6/6LoWPAN, 6TiSCH, RPL and COAP. It also has a large collection of example applications. This means you can easily create a network of IoT devices and have them communicate with each other. If you want to create your own protocol you can simply look at how the other protocols are implemented and use that as a starting point. This makes contiki-ng great in the research world.

In order to run on constrained devices the code footprint is kept low. The OS is on the order of 100 kB while the required RAM can be as low as 10 kB.

1.3.1 CONTIKI-OS

Oikonomou et al., [2022](#) goes over how contiki-ng came to be. Below is a short summary of the article.

Contiki-os was developed in 2003 in order to have an OS build for constrained devices. The main strengths of contiki-os were

- Use of the standard C programming language application developers no longer needed to learn a bespoke language such as nesC.
- Cooperative multi-threading based API. It greatly simplified application programming, reducing the need for both explicit state machines and locks (compared to event-based API and preemptive thread API, respectively).
- Early, standards-compliant support for network protocols such as the Internet Protocol (IP) and IPv6 over Low Power Wireless Personal Area Networks (6LoWPAN) and of the Routing Protocol for Low-Power and Lossy Networks (RPL).
- the cooja simulator for IEEE 802.15.4 networks

However it still had its limitations.

- Large legacy of old, extremely resource-constrained platforms. The 8-bit and 16-bit low-power microcontrollers which contiki was originally designed for became obsolete over time; 32-bit ARM Cortex-M based devices with more sophisticated low-power mode are the new norm.
- Support for non-standard protocols. As the field of wireless sensor network research evolved to become one of the core enabling technologies of the IoT, interoperability and standards became increasingly important. Alongside standards-compliant protocol implementations, the contiki code-base also featured older, experimental, non-standard protocols initially contributed as research artefacts. One such example is the Rime stack.

In order to solve these problems contiki required a big overhaul. In order not to break backwards compatibility a new fork was created, contiki-ng. contiki-ng was created with the following goals in mind:

- focus on dependable (reliable and secure) standard-based IPv6 communication
- Focus on modern IoT platforms, e.g. ARM Cortex M3 and other 32-bit MCUs
- Modernize the structure, configuration, logging and platforms, to reflect the goals above

- Improve the documentation, both code API, module description, and tutorials
- Implement a more agile development process, with easier inclusion of new features, and with periodic releases.

1.3.2 OS FOR THIS PROJECT

For this project we will be using both contiki-ng and contiki-os.

contiki-ng will be used to write the final implementation of the RDC layer. This is because contiki-ng is the newer version of contiki-os and is still being actively developed. It also has a larger community and more documentation.

contiki-os will be used as a reference for how to implement the RDC layer. While there are some changes between contiki-os and contiki-ng some things like the network stack are still largely the same.

1.3.3 CONTIKI-NG DIRECTORY STRUCTURE

In order to add a RDC layer we first need to understand the contiki-ng directory structure in fig. 1.10.

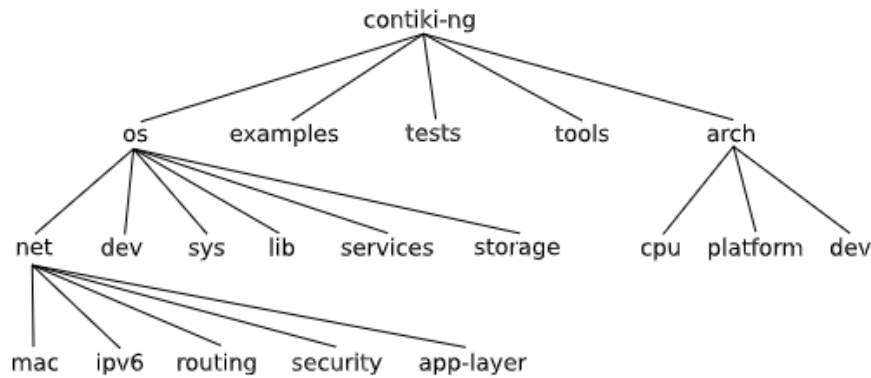


Figure 1.10: contiki-ng directory structure, taken from Oikonomou et al., 2022

- OS: Platform independent code like kernel, software timers, data structure libraries and networking protocols.
- examples: this directory contains many examples which can be directly implemented or used as a basis for your own project.
- tests: automated continuous testing
- tools: contains utilities such as scripts used to upload firmware, generate documentation, COOJA simulator, etc.
- arch: platform specific code to make the OS work on specific hardware. This includes drivers for radio, timers, other on-chip and off-chip peripherals such as UART, SPI, I2C, LED's, buttons and sensors.

Looking at the directory structure, we know we should add the RDC layer into OS/net. For this project we will also be using an external 2.4 GHz LoRa radio. The driver for this radio should be placed in arch/dev/radio.

1.3.4 CONTIKI-NG NETWORK STACK

In fig. 1.11 we can see the contiki-ng network stack. contiki-ng uses 5 layers with each one supporting several protocols. Our goal is to add an additional layer between the MAC layer and the PHY layer. This layer will be the RDC layer.

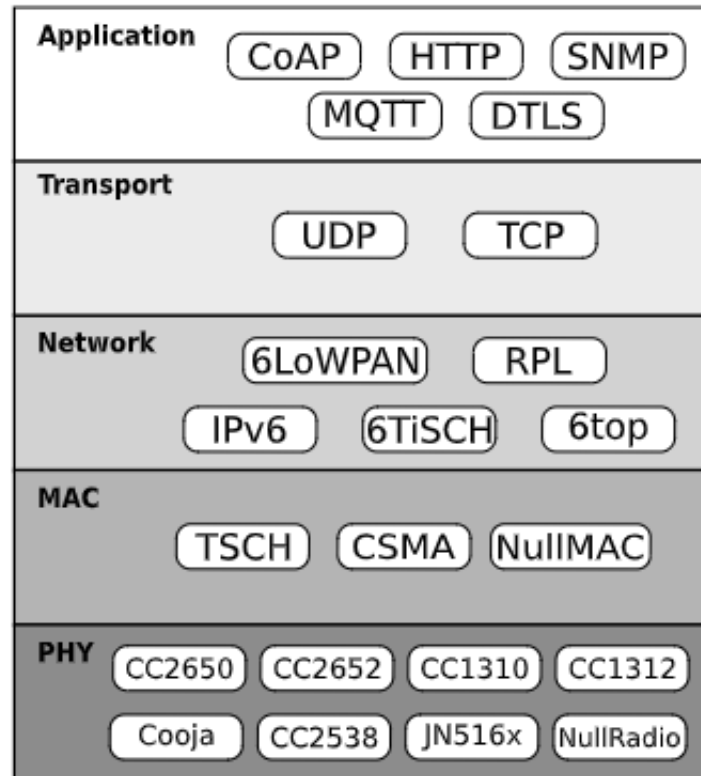


Figure 1.11: contiki-ng network stack, taken from Oikonomou et al., [2022](#)

1.4 ANALASYS PREVIOUS WORK

It is important to note that this project was worked on by previous students. In this section we will take a look at what they have done and how we can use that.

The main part they worked on was creating a driver for the sx1280 2.4 GHz LoRa radio. In order to easily work with the driver they choose to store it outside the contiki-ng directory structure. Since the driver works correctly now i decided to add it into the contiki-ng directory structure. Since it is hardware specific it should be placed under arch/dev/radio. This way we can control the radio being used through the project-conf.h file by adding the line

```
#define NETSTACK_CONF_RADIO sx128x_radio_driver.
```

1.4.1 2.4 GHZ LoRa RADIO DRIVER

The driver contains several files:

- lora24.h: this file contains all the default physical parameters such as SF, bandwidth, preamble length, etc.
- sx128.c: this file contains the functions to initialize the radio, send a packet, receive a packet, etc. These functions use sx128_getset.c to configure the transceiver correctly.
- sx128.h: this file defines the required functions and also contains the settings of the transceiver.
- sx128x_getset.c: This file contains all the functions that are required in order to configure the spi commands that will be send to the transceiver.
- sx128x_internal.c: This file contains the functions that directly communicate with the transceiver. An example of such a function is sx128x_cmd_burst() which is the function that sends the SPI commands to the transceiver.
- sx128x_internal.h: this file contains the definitions of the functions in sx128x_internal.c
- sx128x_registers.h: This file contains the hexadecimal codes for the available commands and settings.

The previous student also added an example although this example is only meant to test the radio driver itself. They chose to create the example in such a way that it directly controls the radio from the code instead of using the netstack of contiki-ng. This means their example cannot be used to test the RDC layer.

While the driver is an important part of the project, it was already finished and tested by the previous students. This means we can assume the driver works correctly and we can focus on the RDC layer. We also don't need an in depth understanding of how the driver works in order to utilize it. For more information on the driver you can look at Peeters, [2022-2023](#).

- Alliance, L. (Nov. 2015). ‘LoRaWAN What Is It’. In: p. 20. URL: <https://loro-alliance.org/wp-content/uploads/2020/11/what-is-lorawan.pdf> (visited on 11/15/2023).
- Benefits of LoRa | Semtech* (2023). URL: <https://www.semtech.com/loro/why-lora> (visited on 11/15/2023).
- Dunkels, A. (Dec. 2011). ‘The ContikiMAC Radio Duty Cycling Protocol’. In: URL: <https://dunkels.com/adam/dunkels11contikimac.pdf>.
- Oikonomou, G., Duquenois, S., Elsts, A., Eriksson, J., Tanaka, Y., and Tsiftes, N. (June 2022). ‘The Contiki-NG Open Source Operating System for next Generation IoT Devices’. In: *SoftwareX* 18, p. 101089. ISSN: 23527110. DOI: [10.1016/j.softx.2022.101089](https://doi.org/10.1016/j.softx.2022.101089). URL: <https://linkinghub.elsevier.com/retrieve/pii/S2352711022000620> (visited on 11/13/2023).
- Peeters, J. (2022–2023). ‘RADIO DUTY CYCLING FOR MULTI-HOP 2.4 GHZ LORA’. In: p. 56.
- Thébaudeau, B. (Sept. 23, 2019). *Radio Duty Cycling*. URL: <https://github.com/contiki-os/contiki/wiki/Radio-duty-cycling>.
- What Are LoRa and LoRaWAN?* (2023). The Things Network. URL: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (visited on 11/13/2023).